PCAP (PCAP-31-03) Study Guide — 4–6 Week Plan (2025)

A practical breakdown of the current PCAP blueprint with checklists, drills, and a week-by-week plan.

Quick Facts

- **Certification:** PCAP™ – Certified Associate in Python Programming
- **Current exam version:** PCAP-31-03
- **Format:** 40 single/multiple-choice questions
- **Time:** 65 minutes (+ ~10 minutes NDA/tutorial)
- **Passing score:** 70%
- **Delivery:** Pearson VUE (test center or OnVUE online)
- **Language:** English

Tip: Aim to finish your first pass by the 40-minute mark; reserve ~25 minutes to review flagged items and multi-select questions.

How to Use This Guide

1. **Scan the Exam Blueprint** (below) and star anything unfamiliar.
2. Follow the **4–6 week plan** (adjust pace to your schedule).
3. Use the **Do-This Drills** for each objective; they mirror common exam traps.
4. Finish with **two timed, 40-question mocks** under exam conditions.

Exam Blueprint → Master Checklist

Section 1 — Modules & Packages (12%)

- [ ] `import` variants: `import`, `from … import …`, aliasing (`as`), and wildcard (`*`)
- [ ] Qualifying nested modules (e.g., `pkg.sub.mod`)
- [ ] `dir()` for discovery; `sys.path` resolution order
- [ ] Standard library focus: `math` (`ceil`, `floor`, `trunc`, `factorial`, `hypot`, `sqrt`)
- [ ] Standard library focus: `random` (`random`, `seed`, `choice`, `sample`)
- [ ] Standard library focus: `platform` (`platform`, `machine`, `processor`, `system`, `version`, `python_implementation`, `python_version_tuple`)
- [ ] User-defined modules & packages: `__name__`, `__pycache__`, public/private vars, `__init__.py`
- [ ] Package layouts & nested packages vs. directory trees; how Python searches packages

Section 2 — Exceptions (14%)

- [ ] `try/except/else` patterns; multiple `except` blocks and tuples (`except (E1, E2)`)
- [ ] Exception hierarchy basics; order of handlers matters
- [ ] Raising errors: `raise`, `raise ex`, assertions (`assert`)
- [ ] Access details with `except E as e`; common attributes (e.g., `.args`)
- [ ] Define & use custom exceptions (derive from `Exception`)

Section 3 — Strings (18%)

- [ ] Character encodings: ASCII, Unicode, UTF-8; escape sequences
- [ ] Built-ins: `ord`, `chr`
- [ ] Indexing, slicing, immutability; iteration; concatenation & repetition
- [ ] Membership operators: `in`, `not in`
- [ ] Methods: `is*` family (`isalpha`, `isdigit`, `isspace`, etc.), `join`, `split`, `index` vs. `find` vs. `rfind`
- [ ] Sorting reminders: `sorted(seq)` returns a **new** list; `list.sort()` sorts **in place** and returns `None`

Section 4 — Object-Oriented Programming (34%)

- [ ] Concepts: class, object, property/attribute, method, encapsulation, inheritance, polymorphism
- [ ] Instance vs. class variables; initialization patterns
- [ ] `__dict__` (objects vs. classes); private name-mangling (`__attr` → `_ClassName__attr`)
- [ ] Methods; the `self` parameter
- [ ] Introspection: `hasattr`, and meta-props `__name__`, `__module__`, `__bases__`
- [ ] Inheritance: single & multiple; `isinstance`; overriding; MRO basics; the “diamond” problem
- [ ] Special methods: overriding `__str__`
- [ ] Constructors & instantiation (`__init__`)

Section 5 — Miscellaneous (22%)

Scope: List Comprehensions, Lambdas, Closures, Basic I/O - [ ] List comprehensions incl. conditional (`if`) and nested comps - [ ] Anonymous functions; `lambda`; passing lambdas to functions - [ ] `map`, `filter` patterns - [ ] Closures: free variables, returning functions - [ ] I/O: terminology (handles/streams), text vs. binary modes - [ ] File ops: `open`, modes (`'r'`, `'w'`, `'a'`, `'b'`, `'t'`), `close`, `read`, `readline`, `readlines`, `write` - [ ] `errno` basics; using `bytearray` as a buffer - [ ] (Best practice) Context managers: `with open(...) as f:`

4–6 Week Study Plan

Pick **4 weeks** for an intensive run (1–2 hrs/day) or **6 weeks** for a steadier pace (45–60 min/day). Swap days as needed.

Week 1 — Modules & Packages + String Fundamentals

Goals: imports, module discovery, math/random/platform, encodings & slicing. - **Day 1–2:** Imports deep dive; write a tiny package with `__init__.py`; print `__name__`; explore `sys.path` by manipulating it. - **Day 3:** `math` drills; compare `ceil/floor/trunc`; when does `factorial` overflow to big ints? - **Day 4:** `random` drills; demonstrate `seed` determinism; pick vs. sample. - **Day 5:** `platform` functions; print a one-line system summary. - **Day 6:** Encoding essentials; `ord/chr`; slicing patterns; `in/not in`. - **Day 7: Timed quiz (20 Q / 30 min)** + error log review.

Week 2 — Strings Methods + Exceptions

Goals: Master string/list methods & exception handling flow. - **Day 1:** `split`, `rsplit`, `join` gotchas; `index` vs `find`; `rfind`. - **Day 2:** Sorting: `sorted` vs `.sort()`; key functions; stability demos. - **Day 3:** `try/except/else` variants; handler ordering; `except (E1, E2)`. - **Day 4:** `raise`; `assert`; capture exception details with `as e`. - **Day 5:** Write two custom exceptions; bubble/catch at different layers. - **Day 6: Mixed drills (20 Q / 30 min)**; re-implement errors you missed. - **Day 7:** Rest/light review.

Week 3 — OOP Foundations

Goals: Classes, instances, class vs. instance vars, methods, `__dict__`. - **Day 1:** Build a `Vector2D` class; inspect instance/class `__dict__`. - **Day 2:** Private members and name mangling; demonstrate access patterns. - **Day 3:** Add `__str__` and convenience methods; practice `hasattr`. - **Day 4:** Inheritance: `Animal → Dog/Cat`; override methods; `isinstance` tests. - **Day 5:** Multiple inheritance mini-lab; observe MRO; diamond example. - **Day 6: OOP quiz (20 Q / 30 min)**. - **Day 7:** Consolidation & notes.

Week 4 — Lambdas, Closures, Comprehensions, and File I/O

Goals: Functional patterns + robust file handling. - **Day 1:** List comprehensions: filters, nested loops; equivalent `for` loops. - **Day 2:** `lambda`, `map`, `filter` with small datasets; readability guidelines. - **Day 3:** Closures: write `make_counter()` and `memoize()`. - **Day 4:** Files: open modes; text vs. binary; `bytearray` buffering. - **Day 5:** Error-safe I/O; try/except around file ops; context managers. - **Day 6: Full mock (40 Q / 65 min)**; mark timing and confidence. - **Day 7:** Review mock; build a “red list” of weak spots.

Optional Week 5–6 — Exam Polish

- **Week 5:** Two more full mocks + targeted drills on red-list topics.
- **Week 6:** Light coding warm-ups; flashcards; sleep well before exam.

Do-This Drills (by Objective)

1) Modules & Packages

- **Import forms:** Given `pkg/a.py` and `pkg/sub/b.py`, write imports that:
- import `b` into `a` three ways (qualified, alias, selective)
- expose only selected names via `pkg/sub/__init__.py`

- `sys.path` **probe:** Print `sys.path`; add a temp dir at runtime; explain lookup order.
- `math`**:** Implement `ceil_floor(x)` returning `(ceil(x), floor(x))`; test with negatives.
- `random`**:** Show that identical seeds → identical sequences across runs.
- `platform`**:** One-liner: `f"{platform.system()} {platform.release()} – {platform.python_implementation()} {platform.python_version()}"`
- **Gotchas:** Avoid `from module import *` in code and mentally note name collisions.

2) Exceptions

- **Ordering:** Write handlers for `LookupError`, `IndexError`, and `Exception`; prove that order changes behavior.
- **Custom:** Define `class NegativeError(Exception): pass`; raise on negative input; catch and print `.args`.
- `assert`**:** Gate a function's precondition with `assert isinstance(x, int)`; toggle with `python -O` to see effect.

3) Strings

- `index` **vs** `find`**:** Trigger `ValueError` with `"abc".index("z")`; compare with `find` (returns `-1`).
- **Slices:** Produce every other char via `s[::2]`; reverse via `s[::-1]`.
- **Normalize whitespace:** `' '.join(s.split())` trick.
- **Sorting:** Show `sorted("cba")` vs `["c","b","a"].sort()`; highlight return values.

4) OOP

- **Class vs instance:** Demonstrate a mutable class var pitfall (e.g., `class C: items=[]`).
- **Name-mangling:** Define `__secret`; access via `_ClassName__secret` for learning only.
- **MRO:** For `class D(B, C)`, print `D.__mro__` and explain resolution.
- `__str__`**:** Add human-readable `__str__` to a custom class and observe `print(obj)`.

5) Misc (Comprehensions, Lambdas, Closures, I/O)

- **Comps:** Nested comp to flatten a 2-D list; add an `if` filter.
- **Lambdas:** Sort a list of tuples by the second element with `key=lambda t: t[1]`.
- **Closures:** Write `adder(n)` returning a function adding `n`.
- **I/O:** Read a binary file into a `bytearray`, transform bytes, write back.

Common Pitfalls & Exam Traps

- `sorted` **vs** `.sort()`: new list vs in-place; `.sort()` returns `None`.
- `index` **vs** `find`: `index` raises, `find` returns `-1`.
- **Identity vs equality**: `is` / `is not` vs `==` / `!=`.
- **Handler order**: catch child exceptions **before** parent types.
- **Shadowing**: wildcard imports or reusing names hide built-ins and module symbols.
- **Immutability**: string methods return new objects; lists are mutable.

- **Multiple inheritance**: know that MRO resolves left-to-right; diamonds are resolved via C3 linearization.
- **File modes**: forgetting `'b'` for binary causes encoding surprises.
- **Random seeds**: expecting different results without reseeding.

Timed Practice Structure

- **Daily micro-drill** (10–15 min): 10 questions on one topic.
- **Every 3–4 days**: 20 questions / 30 minutes mixed topics.
- **Weekly**: one 40-question full mock / 65 minutes.
- **Review loop**: For each miss, write a 2–3 line "fix note" and a 3–5 line code snippet demonstrating the correct concept.

Exam-Day Strategy & Checklist

Strategy 1. **First pass**: answer what you know, flag the rest. 2. **Second pass**: tackle flagged items, especially multi-select. 3. **Sanity sweep**: look for trick wording, off-by-one slices, handler order.

Checklist - [ ] Two full mocks ≥ 70% under time - [ ] Re-read notes on exceptions, string methods, and OOP MRO - [ ] Quick brain-warm: list comprehension + `try/except` + mini-class - [ ] Stable test environment; ID ready (for OnVUE: room cleared, stable internet)

"One-Pager" Quick Reference (copy to your notes)

Imports - `from m import n as alias` • `dir(obj)` • `sys.path`

Math: `ceil`, `floor`, `trunc`, `factorial`, `hypot`, `sqrt`

Random: `random`, `seed`, `choice`, `sample`

Platform: `platform`, `machine`, `processor`, `system`, `version`, `python_implementation`, `python_version_tuple`

Exceptions - `try/except/else` • order matters • `raise` • `assert` • `except E as e` • custom exceptions

Strings - `ord/chr` • `in/not in` • `split/join` • `index/find/rfind` • slicing `s[a:b:c]`

OOP - `__dict__` • name-mangling `__x` → `_Class__x` • `hasattr` • `__name__`, `__module__`, `__bases__` • `isinstance` • override `__str__`

Misc - List comprehensions (with `if`) • `lambda` • `map`, `filter` • closures • `open`/modes • `read`, `readline`, `readlines`, `write` • `bytearray` • `errno`

Recommended Prep Resources (use any two)

- OpenEDG Python Institute **Python Essentials – Part 2 (Intermediate)** (free, PCAP-aligned)
- Official **PCAP Exam Syllabus** (read cover-to-cover)
- A reputable PCAP practice exam set for timing & confidence tracking
- Your own **mini-project** (100–150 lines) touching: imports, exceptions, OOP, file I/O, and a couple of lambdas/comprehensions

---

Personalization Notes

- If you've shipped tools already, practice converting a script you wrote into a small package with `__init__.py`, unit-test a custom exception, and add a simple CLI that reads/writes files. This hits **every** scoring domain while staying practical.

Good luck! When you can explain `index` *vs* `find`*, draw an MRO, and write a nested comprehension from memory, you're PCAP-ready.*